

Database Design

LearnHub uses **Entity Framework Core 10 with SQLite**, designed code first: the C# entity classes are the source of truth, and migrations generate the schema. There is a single provider, so there is one migration set and one database file — no provider-specific code paths anywhere in the application.

For the entity relationship diagram see [ERD.md](#). For how the database file is persisted in production see [DEPLOYMENT.md](#).

1. Why SQLite

Reason	Detail
Simple to deploy	One file, no separate database service to provision, secure or pay for.
Simple to explain	A single <code>learnhub.db</code> file holds the whole system, which is easy to demonstrate and to describe in the report.
No vendor lock-in	The assignment forbids Azure, and SQLite runs anywhere — a laptop, a container or a Railway volume.
Sufficient for the scale	A single-instance academic deployment has one writer at a time; SQLite handles that comfortably.
Identical in test and production	The integration tests run the real application against SQLite (in memory), so the tested engine is the deployed engine.

SQLite has one write lock for the whole database. That is a deliberate, acceptable trade-off for this project and is recorded as a known limitation — a multi-instance deployment would need a client/server database.

2. Where the database lives

The connection string is resolved in `src/LearnHub/Data/DatabaseServiceCollectionExtensions.cs`:

1. The environment variable `DATABASE_CONNECTION_STRING` wins when it is set. Railway supplies it.
2. Otherwise `ConnectionStrings:DefaultConnection` from `appsettings.json` is used, which defaults to `Data Source=App_Data/learnhub.db`.

A relative path is resolved against the content root, and the containing folder is created automatically, so the database location never depends on the current working directory.

Environment	Connection string	Notes
Local development	<code>Data Source=App_Data/learnhub.db</code>	Created on first run. Delete <code>App_Data/</code> to start again.

Environment	Connection string	Notes
Integration tests	Data Source=...;Mode=Memory;Cache=Shared	One private in-memory database per test factory, so tests never touch a developer's data.
Railway (production)	Data Source=/data/learnhub.db	/data is the mounted volume, so the file survives a redeploy.

`Database:ApplyMigrationsOnStartup` (default `true`) applies pending migrations during start-up. On the single-instance deployment used here that is simpler and more reliable than a separate migration step. If it is set to `false` and migrations are pending, the application logs a warning and skips seeding rather than running against an out-of-date schema.

3. Schema at a glance

One migration — `src/LearnHub/Data/Migrations/20260914100832_InitialCreate.cs` — creates **18 tables**: eleven application tables and the seven tables ASP.NET Core Identity needs.

Application tables

Table	Purpose	Key columns
Categories	Subject areas that group courses	<code>Id</code> , <code>Name</code> (unique), <code>Description</code> , <code>IconName</code>
Courses	The catalogue	<code>Id</code> , <code>CategoryId</code> → <code>Categories</code> , <code>Title</code> , <code>ShortDescription</code> , <code>Description</code> , <code>LearningOutcomes</code> , <code>InstructorName</code> , <code>Difficulty</code> , <code>DurationMinutes</code> , <code>ThumbnailPath</code> , <code>IsPublished</code>
LearningResources	The lessons inside a course	<code>Id</code> , <code>CourseId</code> → <code>Courses</code> , <code>Title</code> , <code>Summary</code> , <code>Type</code> , <code>Body</code> , <code>Solution</code> , <code>ExternalUrl</code> , <code>FilePath</code> / <code>FileName</code> / <code>FileContentType</code> / <code>FileSizeBytes</code> , <code>EstimatedMinutes</code> , <code>SortOrder</code> , <code>IsPreview</code> , <code>IsPublished</code>
Enrollments	A student's membership of a course	<code>Id</code> , <code>UserId</code> → <code>AspNetUsers</code> , <code>CourseId</code> → <code>Courses</code> , <code>EnrolledAt</code> , <code>LastAccessedAt</code> , <code>CompletionPercentage</code> , <code>CompletedAt</code>
ResourceCompletions	Which lessons a student has finished	<code>Id</code> , <code>UserId</code> → <code>AspNetUsers</code> , <code>LearningResourceId</code> → <code>LearningResources</code> , <code>CompletedAt</code>
Quizzes	A quiz attached to a course	<code>Id</code> , <code>CourseId</code> → <code>Courses</code> , <code>Title</code> , <code>Description</code> , <code>PassMarkPercent</code> , <code>IsPublished</code>
Questions	A quiz question	<code>Id</code> , <code>QuizId</code> → <code>Quizzes</code> , <code>Text</code> , <code>Explanation</code> , <code>Points</code> , <code>SortOrder</code>

Table	Purpose	Key columns
AnswerOptions	The selectable answers to a question	Id, QuestionId → Questions, Text, IsCorrect, SortOrder
QuizAttempts	One submitted attempt	Id, QuizId → Quizzes, UserId →AspNetUsers, StartedAt, CompletedAt, CorrectCount, QuestionCount, Score, MaxScore, ScorePercent, Passed
QuizAnswers	The answer chosen for one question in one attempt	Id, QuizAttemptId → QuizAttempts, QuestionId → Questions, SelectedOptionId → AnswerOptions (nullable), IsCorrect
ContactMessages	Enquiries from the public contact form	Id, Name, Email, Subject, Message, IsRead, CreatedAt

Identity tables

AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims, AspNetRoleClaims, AspNetUserLogins, AspNetUserTokens.

ApplicationUser extends IdentityUser with FullName, Bio and CreatedAt. Passwords are hashed by ASP.NET Core Identity and are never stored in plain text.

Quiz attempt history is a snapshot. QuizAttempt stores CorrectCount, QuestionCount, Score, MaxScore and ScorePercent as they were at submission time, so editing a quiz later does not rewrite a student's past results.

4. Relationships

Relationship	Cardinality	Delete rule
Category → Course	one-to-many	Restrict — a category holding courses cannot be deleted
Course → LearningResource	one-to-many	Cascade
Course → Quiz	one-to-many	Cascade
Course → Enrollment	one-to-many	Cascade

Relationship	Cardinality	Delete rule
ApplicationUser → Enrollment	one-to-many	Cascade
ApplicationUser → ResourceCompletion	one-to-many	Cascade
LearningResource → ResourceCompletion	one-to-many	Cascade
Quiz → Question	one-to-many	Cascade
Question → AnswerOption	one-to-many	Cascade
Quiz → QuizAttempt	one-to-many	Cascade
ApplicationUser → QuizAttempt	one-to-many	Cascade
QuizAttempt → QuizAnswer	one-to-many	Cascade
Question → QuizAnswer	one-to-many	Restrict
AnswerOption → QuizAnswer	one-to-many	Restrict

The two **Restrict** rules on `QuizAnswer` are deliberate: they stop an administrator editing a quiz from silently destroying a student's answer history. The management services clear or delete dependent answers explicitly before removing a question or an option, so the administrator still gets a clear message instead of a database error.

5. Constraints that keep the data honest

Unique indexes

Index	Table	Prevents
Name	Categories	Two categories with the same name
(UserId, CourseId)	Enrollments	Duplicate enrolment — a student cannot enrol twice in one course
(UserId, LearningResourceId)	ResourceCompletions	The same lesson being marked complete twice

Index	Table	Prevents
(QuizAttemptId, QuestionId)	QuizAnswers	Two answers to the same question in one attempt

The application checks for duplicates first and shows a friendly message; the unique index is the guarantee that the rule holds even if two requests race.

Check constraints

Constraint	Table	Rule
CK_Courses_DurationMinutes	Courses	DurationMinutes > 0
CK_LearningResources_SortOrder	LearningResources	SortOrder >= 0
CK_Enrollments_CompletionPercentage	Enrollments	CompletionPercentage BETWEEN 0 AND 100
CK_Quizzes_PassMarkPercent	Quizzes	PassMarkPercent BETWEEN 0 AND 100
CK_Questions_Points	Questions	Points BETWEEN 1 AND 100
CK_QuizAttempts_ScorePercent	QuizAttempts	ScorePercent BETWEEN 0 AND 100
CK_QuizAttempts_CorrectCount	QuizAttempts	CorrectCount BETWEEN 0 AND QuestionCount
CK_QuizAttempts_Score	QuizAttempts	Score BETWEEN 0 AND MaxScore

Required columns are enforced with `IsRequired()`, string lengths come from `src/LearnHub/Models/FieldLengths.cs`, and genuinely optional values (`Bio`, `ThumbnailPath`, `CompletedAt`, `SelectedOptionId`) are nullable.

Enumerations

`Difficulty` and `ResourceType` are stored as **strings** (`HasConversion<string>()`), so the database is readable and a reordering of the C# enum cannot silently change the meaning of existing rows.

Indexes for the pages that are actually used

`Courses(Title)`, `Courses(IsPublished, CategoryId)`, `LearningResources(CourseId, SortOrder)`, `LearningResources(CourseId, IsPublished)`, `Enrollments(CourseId, CompletionPercentage)`, `Questions(QuizId, SortOrder)`, `AnswerOptions(QuestionId, SortOrder)`, `QuizAttempts(UserId, QuizId)`, `QuizAttempts(CompletedAt)`.

Each one backs a real query — catalogue search and filtering, the lesson list, progress calculation, the quiz player and the admin dashboards.

6. Seeding

`src/LearnHub/Data/Seed/DatabaseInitializer.cs` runs once at start-up, after migrations:

1. Creates the `Admin` and `Student` roles.
2. Creates the administrator from `Seed:AdminEmail` / `Seed:AdminPassword` if that account does not exist. An existing account is never modified — an administrator may have changed their own password deliberately.
3. Seeds the demonstration catalogue via `DemoDataSeeder` when `Seed:DemoData` is true.

The seeded catalogue is defined in `Data/Seed/DemoCatalog*.cs` and contains **8 categories, 12 courses (11 published and 1 draft), 64 learning resources, 11 quizzes, 54 questions, 216 answer options and 7 learners**. Extra learners and staggered enrolment dates make the dashboards and progress pages show realistic figures rather than empty states.

Demonstration accounts — DEMO ONLY

Role	Email
Administrator	<code>admin@learnhub.local</code>
Student	<code>demo.student@example.com</code>

No password is ever committed. In Development, `DevelopmentSeedPasswords` generates strong random passwords on the first run and writes them to `App_Data/demo-credentials.json`, which is git-ignored. In Production the passwords must come from the `Seed__AdminPassword` and `Seed__DemoStudentPassword` environment variables; with no password set the account still exists (so seeded data stays realistic) but cannot be signed into, and the log explains what to configure. These accounts exist for demonstration and must not be used for anything real.

7. Working with the database

```
# Restore the EF Core tools pinned in .config/dotnet-tools.json
dotnet tool restore

# Create a migration after changing a model
dotnet ef migrations add <Name> --project src/LearnHub --output-dir Data/Migrations

# Apply migrations by hand (the application also does this at start-up)
dotnet ef database update --project src/LearnHub

# Inspect the generated SQL
dotnet ef migrations script --project src/LearnHub --output schema.sql
```

The `EF Core migration (cloud)` GitHub workflow (`.github/workflows/ef-migrations.yml`) can add a migration from the browser, without the SDK installed.

`src/LearnHub/Data/DesignTimeDbContextFactory.cs` supplies the context to the `dotnet ef` tools so migrations can be created without starting the web application.

Backups

The whole database is one file. Copy `/data/learnhub.db` from the Railway volume to back it up, and copy it back to restore. Because migrations are applied at start-up, restoring an older file and restarting the service brings the schema up to date automatically.

8. Verifying the schema

- `tests/LearnHub.Tests/Data/DatabaseModelTests.cs` asserts the constraints, indexes and delete rules described above, so the model cannot drift from this document without a test failing.
- `tests/LearnHub.Tests/Infrastructure/TestDatabase.cs` creates a fresh in-memory SQLite database per factory and applies the real migrations, so the migrations themselves are exercised by every integration test.